

Gauss

高斯消元

引入

高斯消元法（Gauss–Jordan elimination）是求解线性方程组的经典算法，它在当代数学中有着重要的地位和价值，是线性代数课程教学的重要组成部分。

高斯消元法除了用于线性方程组求解外，还可以用于行列式计算、求矩阵的逆，以及其他计算机和工程方面。

消元法及高斯消元法思想

定义

消元法是将方程组中的一方程的未知数用含有另一未知数的代数式表示，并将其带入到另一方程中，这就消去了一未知数，得到一解；或将方程组中的一方程倍乘某个常数加到另外一方程中去，也可达到消去一未知数的目的。消元法主要用于二元一次方程组的求解。

解释

例一：利用消元法求解二元一次线性方程组：

解：将方程组中两方程相加，消元 可得：

解得：

将 代入方程组中第二个方程可得：

消元法理论的核心

消元法理论的核心主要如下：

- 两方程互换，解不变；
- 一方程乘以非零数，解不变；
- 一方程乘以数 加上另一方程，解不变。

高斯消元法思想概念

德国数学家高斯对消元法进行了思考分析，得出了如下结论：

- 在消元法中，参与计算和发生改变的是方程中各变量的系数；
- 各变量并未参与计算，且没有发生改变；

- 可以利用系数的位置表示变量，从而省略变量；
- 在计算中将变量简化省略，方程的解不变。

高斯在这些结论的基础上，提出了高斯消元法，首先将方程的增广矩阵利用行初等变换化为行最简形，然后以线性无关为准则对自由未知量赋值，最后列出表达方程组通解。

高斯消元五步骤法

解释

高斯消元法在将增广矩阵化为最简形后对于自由未知量的赋值，需要掌握线性相关知识，且赋值存在人工经验的因素，使得在学习过程中有一定的困难，将高斯消元法划分为五步骤，从而提出五步骤法，内容如下：

1. 增广矩阵行初等行变换为行最简形；
2. 还原线性方程组；
3. 求解第一个变量；
4. 补充自由未知量；
5. 列表示方程组通解。

利用实例进一步说明该算法的运作情况。

过程

例二：利用高斯消元法五步骤法求解线性方程组：

增广矩阵行（初等）变换为行最简形

所谓增广矩阵，即为方程组系数矩阵 与 常数列 的并生成的新矩阵，即，增广矩阵行初等变换化为行最简形，即是利用了高斯消元法的思想理念，省略了变量而用变量的系数位置表示变量，增广矩阵中用竖线隔开了系数矩阵和常数列，代表了等于符号。

化为行阶梯形

化为最简形

还原线性方程组

解释

所谓的还原线性方程组，即是在行最简形的基础上，将之重新书写为线性方程组的形式，即将行最简形中各位置的系数重新赋予变量，中间的竖线还原为等号。

求解第一个变量

解释

即是对于所还原的线性方程组而言，将方程组中每个方程的第一个变量，用其他量表达出来。如方程组两方程中的第一个变量 x_1 和 x_2

补充自由未知量

解释

第 3 步中，求解出变量 x_1 和 x_2 ，从而说明了方程剩余的变量 x_3 和 x_4 不受方程组的约束，是自由未知量，可以取任意值，所以需要在第 3 步骤解得基础上进行解得补充，补充的方法为 $x_3 = x_3$ ，这种解得补充方式符合自由未知量定义，并易于理解，因为是自由未知量而不受约束，所以只能自己等于自己。

列表示方程组的通解

其中 c_1 和 c_2 为任意常数。

解释

即在第 4 步的基础上，将解表达为列向量组合的表示形式，同时由于 x_3 和 x_4 是自由未知量，可以取任意值，所以在解得右边，令二者分别为任意常数 c_1 和 c_2 ，即实现了对方程组的求解。

行列式计算

解释

方阵行列式（Determinant）可以理解为所有列向量所夹的几何体的有向体积

例如：

行列式有公式

其中 S_n 是指长度为 n 的全排列的集合， σ 就是一个全排列，如果 σ 的逆序对对数为偶数，则 $\text{sgn}(\sigma) = 1$ ，否则 $\text{sgn}(\sigma) = -1$ 。

通过体积概念理解行列式不变性是一个非常简单的办法：

- 矩阵转置，行列式不变；
- 矩阵行（列）交换，行列式取反；
- 矩阵行（列）相加或相减，行列式不变；
- 矩阵行（列）所有元素同时乘以数 k ，行列式等比例变大。

由此，对矩阵应用高斯消元之后，我们可以得到一个对角线矩阵，此矩阵的行列式由对角线元素之积所决定。其符号可由交换行的数量来确定（如果为奇数，则行列式的符号应颠倒）。因此，我们可以在 n 的复杂度下使用高斯算法计算矩阵。

注意，如果在某个时候，我们在当前列中找不到非零单元，则算法应停止并返回 0。

实现

```
1 constexpr double EPS = 1E-9;
2 int n;
3 vector<vector<double>> a(n, vector<double>(n));
4
5 double det = 1;
6 for (int i = 0; i < n; ++i) {
7     int k = i;
8     for (int j = i + 1; j < n; ++j)
9         if (abs(a[j][i]) > abs(a[k][i])) k = j;
10    if (abs(a[k][i]) < EPS) {
11        det = 0;
12        break;
13    }
14    swap(a[i], a[k]);
15    if (i != k) det = -det;
16    det *= a[i][i];
17    for (int j = i + 1; j < n; ++j) a[i][j] /= a[i][i];
18    for (int j = 0; j < n; ++j)
19        if (j != i && abs(a[j][i]) > EPS)
20            for (int k = i + 1; k < n; ++k) a[j][k] -= a[i][k] * a[j][i];
21 }
22
23 cout << det;
```

矩阵求逆

对于方阵，若存在方阵，使得，则称矩阵可逆，被称为它的逆矩阵。

给出阶方阵，求解其逆矩阵的方法如下：

1. 构造的矩阵；
2. 用高斯消元法将其化简为最简形，即可得到的逆矩阵。如果最终最简形的左半部分不是单位矩阵，则矩阵不可逆。

该方法的正确性证明需要用到较多线性代数的知识，限于篇幅这里不再给出。感兴趣的读者可以自行查阅相关资料。

高斯消元法解异或方程组

异或方程组是指形如

的方程组，其中表示「按位异或」（即 `xor` 或 C++ 中的 `^`），且式中所有系数/常数（即与）均为 0 或 1。

由于「异或」符合交换律与结合律，故可以按照高斯消元法逐步消元求解。值得注意的是，我们在消元的时候应使用「异或消元」而非「加减消元」，且不需要进行乘除改变系数（因为系数均为 0 和 1）。

注意到异或方程组的增广矩阵是矩阵（矩阵中仅含有 0 与 1），所以我们可以使用 C++ 中的 `std::bitset` 进行优化，将时间复杂度降为 $O(n^2 \log n)$ ，其中 n 为元的个数， m 为方程条数，一般为 $O(n)$ （与机器有关）。

参考实现：

```

1 std::bitset<1010> matrix[2010]; // matrix[1~n]: 增广矩阵, 0 位置为常数
2
3 std::vector<bool> GaussElimination(
4     int n, int m) // n 为未知数个数, m 为方程个数, 返回方程组的解
5     // (多解 / 无解返回一个空的 vector)
6 {
7     for (int i = 1; i <= n; i++) {
8         int cur = i;
9         while (cur <= m && !matrix[cur].test(i)) cur++;
10        if (cur > m) return std::vector<bool>(0);
11        if (cur != i) swap(matrix[cur], matrix[i]);
12        for (int j = 1; j <= m; j++)
13            if (i != j && matrix[j].test(i)) matrix[j] ^= matrix[i];
14    }
15    std::vector<bool> ans(n + 1);
16    for (int i = 1; i <= n; i++) ans[i] = matrix[i].test(0);
17    return ans;
18 }

```

练习题

- [Codeforces - 巫师和赌注](#)
- [luogu - SDOI2010 外星千足虫](#)

本页面最近更新: 2024/10/9 22:38:42, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [StudyingFather](#), [CCXXXI](#), [Chrogeek](#), [ChungZH](#), [countercurrent-time](#), [Early0v0](#), [Enter-tainer](#), [GavinZhengOI](#), [Great-designer](#), [H-J-Granger](#), [henrytbtrue](#), [HeRaNO](#), [huayucaiiji](#), [iamtwz](#), [lr1d](#), [ksyx](#), [MegaOwler](#), [NachtgeistW](#), [P-Y-Y](#), [qwqAutomaton](#), [shuzhouliu](#), [shuzhouliu-bot](#), [sshwy](#), [SukkaW](#), [Tiphereth-A](#), [tsentau](#), [WhenMelancholy](#), [Xeonacid](#), [Yukimaikoriya](#), [Zhoier](#), [Siger Young](#), [Siger Young](#), [YanJun-Zhao](#), [zyj-111](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供, 附加条款亦可能应用